```
    async(FunctionType&& func,Args&& ... args);

    template<typename FunctionType,typename ... Args>
    future<result_of<FunctionType(Args...)>::type>
    async(std::launch policy,FunctionType&& func,Args&& ... args);

}
```

D.4.1. std::future class template

The `std::future` class template provides a means of waiting for an asynchronous result from another thread, in conjunction with the `std::promise` and `std::packaged_task` class templates and the `std::async` function template, which can be used to provide that asynchronous result. Only one `std::future` instance references any given asynchronous result at any time.

Instances of `std::future` are `MoveConstructible` and `MoveAssignable` but not `CopyConstructible` or `CopyAssignable`.

Class definition

```
template<typename ResultType>
class future
{
public:
    future() noexcept;
    future(future&&) noexcept;
    future& operator=(future&&) noexcept;
    ~future();

    future(future const&) = delete;
    future& operator=(future const&) = delete;

    shared_future<ResultType> share();

    bool valid() const noexcept;

    see description get();

    void wait();

    template<typename Rep,typename Period>
```